

사내 연차 경매 시스템

Annual Leave Auction System — 요구분석 및 설계 결과

팀 타임소프트콘 · 김기철 · 오지석

고연차의 *버려지는 연차* 와 저연차의 *부족한 연차* 를
B2E Escrow & Dividend 아키텍처로 잇는,
*사내 금융 수준 정합성*을 갖춘 연차 거래 플랫폼

① 프로젝트 개요

해결하려는 문제 · 사용자 · 핵심 기능

① 문제 정의 — 사내 "연차 양극화"

집단	상황	손해
고연차 직원	업무 과다로 연차를 다 못 씀	연말에 미사용 연차 소멸 (0원)
저연차 직원	법정 연차가 적음	휴가 부족 호소

- 양쪽 모두 손해인 현상이 매년 반복 → 사내 복지 비효율
- 단순 아이디어: "직원끼리 연차를 사고팔자" (P2P)
 - 폐기: 근로기준법 제60·61조 — 휴식권의 매매는 위법
 - → 개인 간 직접 거래(P2P)·현금 결제는 설계에서 **영구 배제**

핵심 도전 과제: **합법적이면서도 회사 재무 리스크 0%**인 매칭 구조를 어떻게 만들 것인가?

① 해결책 — B2E Escrow & Dividend

수익적립금 = 구매자가 낸 포인트를 회사가 못 쓰게 따로 모아두는 공동의 돈.
연말에 이 모인 돈만큼만 판매자에게 나눠줍니다.
중고거래 안전결제처럼 모아뒀다 정산 — 영문 아키텍처명에선 Escrow.

회사가 중개하고(B2E), 대가는 복지 포인트, 정산은 연말 일괄 배당.

전환 축	초기 아이디어	최종 설계
거래 주체	직원 — 직원 (P2P)	회사가 중개 (B2E)
재화	현금(KRW)	사내 복지 포인트
정산 시점	즉시 지급	연말 일괄 배당
매물 단위	날짜 종속 연차	"연차 1일권" (개수, 날짜 비종속)

- 구매자가 낸 포인트는 회사 예산이 아니라 수익적립금으로 차곡차곡 쌓임
- 연말에 판매자(기여자)에게 지분(Stake = 기여한 연차의 비율)대로 복지카드 한도로 배당
- 회사는 예산 선투입 0원 → 수익적립금에 모인 잔액 안에서만 배당 (재무 리스크 0%)

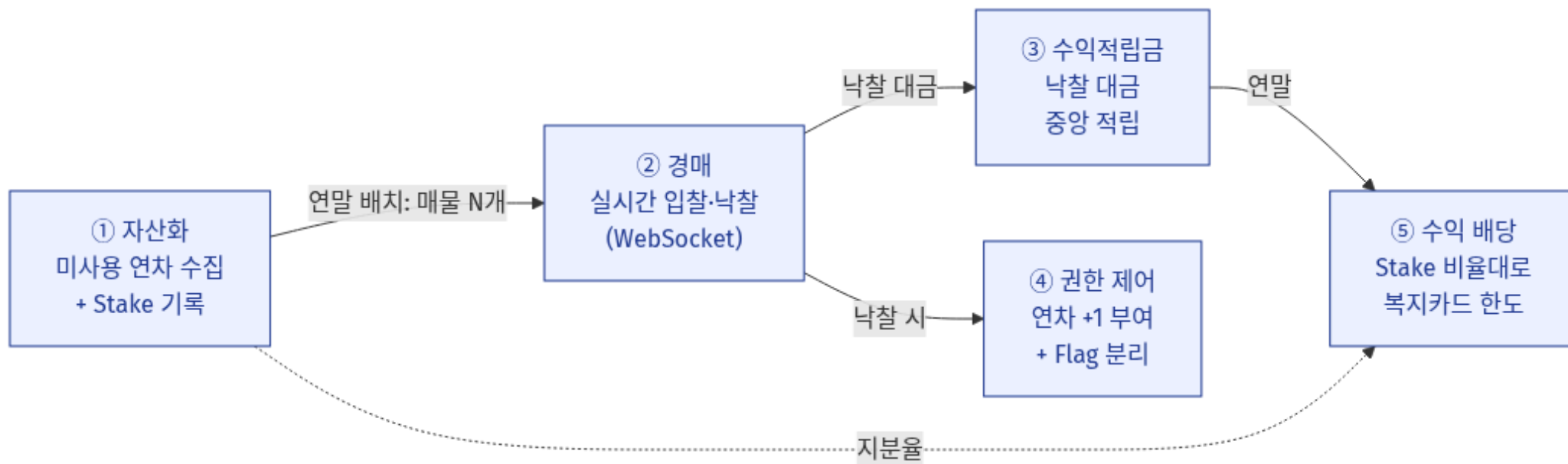
① 사용자(액터)와 3-Way Win

사용자 = 행위 기반 역할: 직원(공통 base)에서 판매자 · 구매자 · 관리자 역할이 파생
외부 액터: 스케줄러(12/31 배치) · HR/ezpass 시스템

참여자	얻는 가치
회사/HR	예산 0원, 재무 리스크 0% — 비정규 연차는 수당 정산에서 제외
판매자(고연차)	소멸될 0원 연차 → 연말 배당금(복지 한도)
구매자(저연차)	포인트로 부족한 연차 확보, 원하는 시기 휴식

동일 직원이 시점별로 판매자·구매자를 **동시 수행** 가능.

① 핵심 기능 — 5개 하위 시스템



② 요구사항 분석

기능 / 비기능 + 우선순위

② 기능 요구사항 (FR) + 우선순위

핵심 도메인은 연차 경매 — 입찰·낙찰이 시스템의 심장이고, 나머지는 그 결과를 반영·정산하는 지원 기능.

(우선순위 MoSCoW: M=Must / S=Should / C=Could)

영역	기능	우선순위
경매	FR-2.1 실시간 입찰 — 포인트 한도 내, 상위 입찰 시 WebSocket 알림	M
경매	FR-2.2 낙찰 · 수익적립금 적립 — 포인트 차감 → 중앙 대장 누적	M
낙찰 결과 반영	FR-2.3 연차 부여(AUCTION +1) · FR-3.1 차감 우선순위(AUCTION→EVENT→REGULAR)	M
연말 배치	FR-1.1 공용 풀 생성(REGULAR 수집 + Stake) · FR-4.1 배당 정산(Stake 비례)	M
부가	FR-5.2 잔액·내역 조회 · FR-5.1 관리자 적립 · FR-4.2 유찰 EVENT 지급	S / C

② 비기능 요구사항 (NFR) — "사내 금융급 정합성"

ID	비기능 요구	실현 방법
NFR-1	동시성 — 마감 직전 동시 입찰 Race 0 건	SQLite write 락 (lockAuction no-op UPDATE)으로 같은 경매 입찰 직렬화
NFR-2	재무 정합성·감사 추적 — 배당 오차 0원	거래 기록 Insert-Only + 수익적립금 등식 통화별 검증
(보안)	인증·권한	중앙 인증(ezpass) 위임 + JWT, RBAC(EMPLOYEE/ADMIN)
(실시간)	입찰가·타이머 즉시 반영	WebSocket 브로드캐스트

재무 무결성 (NFR-2 핵심):

모든 포인트 이동(입찰·낙찰·환불·배당)을 장부에 기록 →
들어온 포인트 - 나간 포인트 = 수익적립금 잔액 (항상 일치, 통화별)
 ⇒ 배당이 모은 돈을 **1포인트도 초과 못 함** (회사 손실 0)

② 무결성 제약 (DB-RULE) — 설계를 떠받치는 규칙

요구사항을 "깨지면 안 되는 규칙"으로 못 박은 4가지:

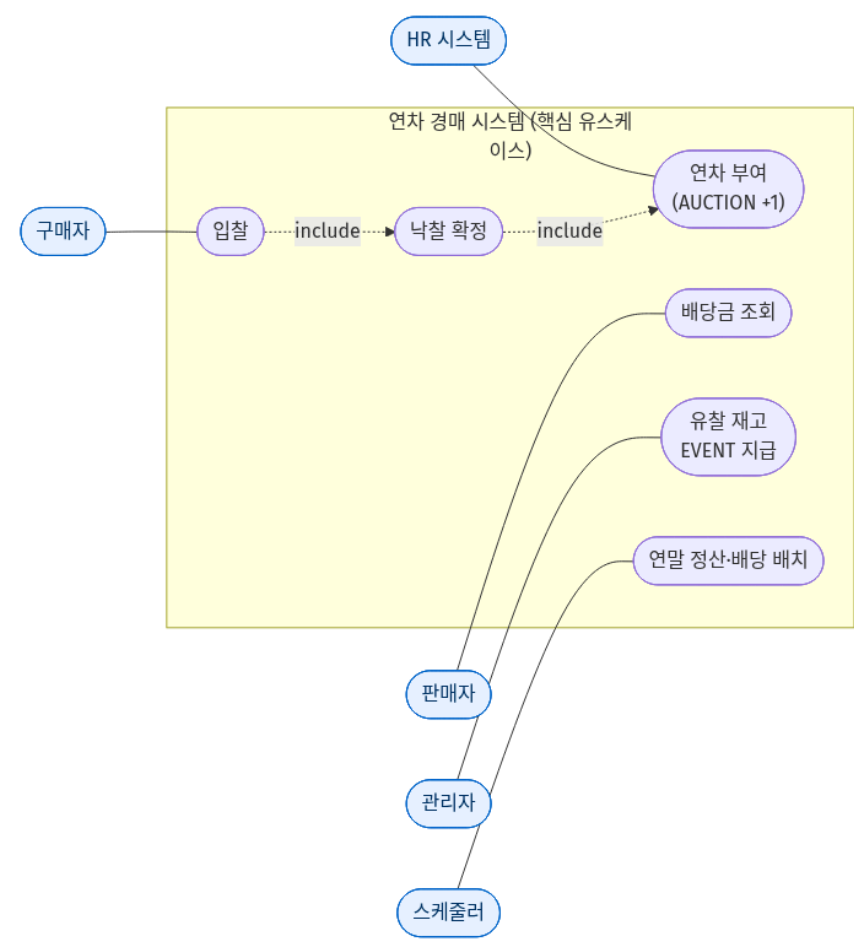
1. **거래 기록 불변 (Insert-Only)** — LEDGER_ENTRY 는 INSERT만, UPDATE/DELETE는 DB 트리거로 차단.
환불도 새 보정 INSERT.
2. **3-Flag 분리** — REGULAR (수당 O) / AUCTION (수당 X) / EVENT (수당 X) → 이중 보상(Double-Dipping) 사고 방지
3. **차감 우선순위 강제** — AUCTION → EVENT → REGULAR (사용자 선택 불가)
4. **수익적립금 상한** — 통화별 배당 합 = 수익적립금 잔액 (초과 금지)

이 시스템은 연차 게시판이 아니라 사내 복지 예산·HR 정산이 직결된 플랫폼 → 무결성이 1순위 요구사항.

③ UML 모델링

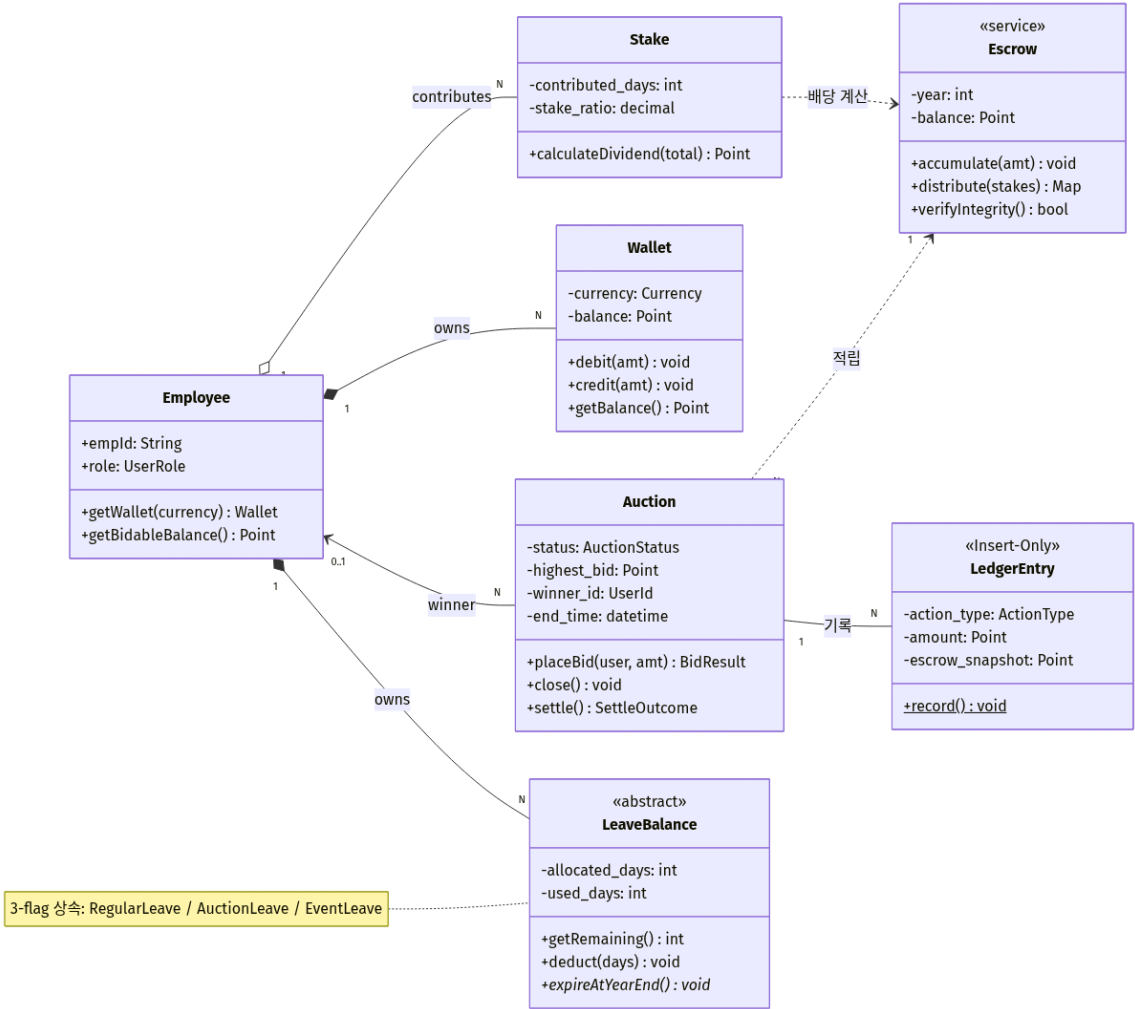
기본 4종(유스케이스·클래스·순차·상태) + 심화 3종(활동·컴포넌트·객체)

③-1 유스케이스 다이어그램

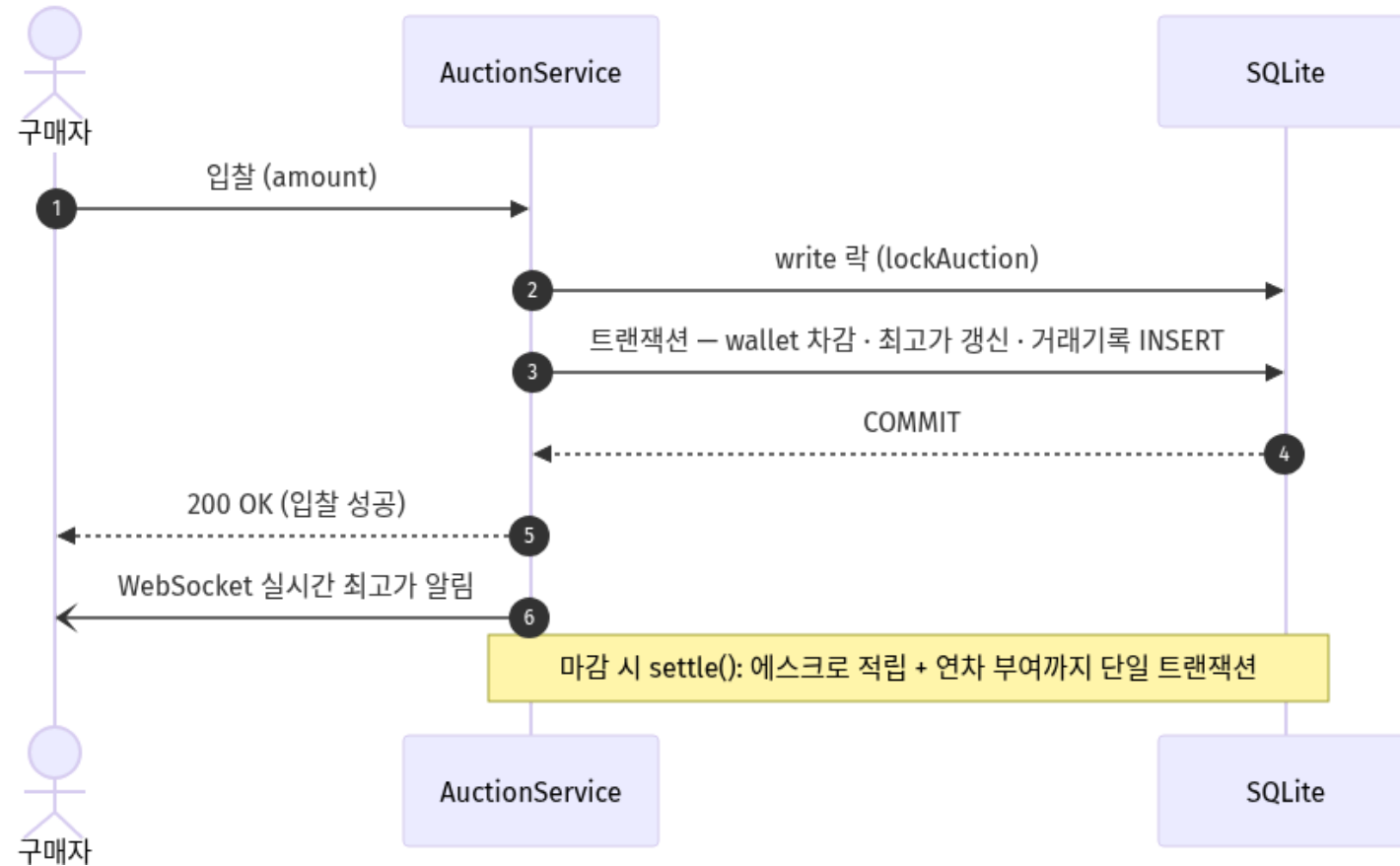


핵심 유스케이스 — 입찰 → 낙찰 → 연차부여(«include»). · 전체 docs/uml

③-2 클래스 다이어그램

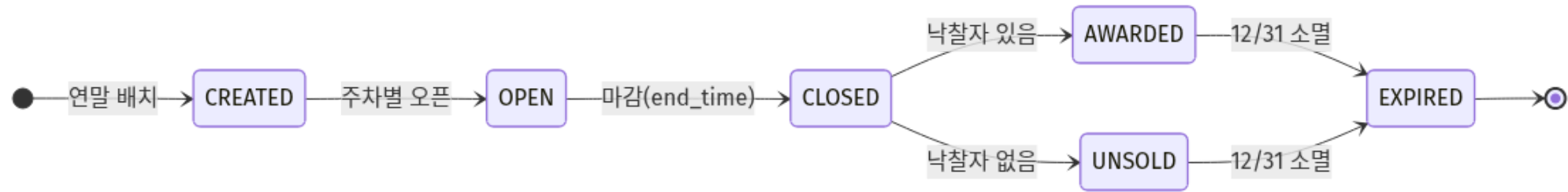


③-3 순차 다이어그램 — 입찰 → 낙찰 → 연차 부여



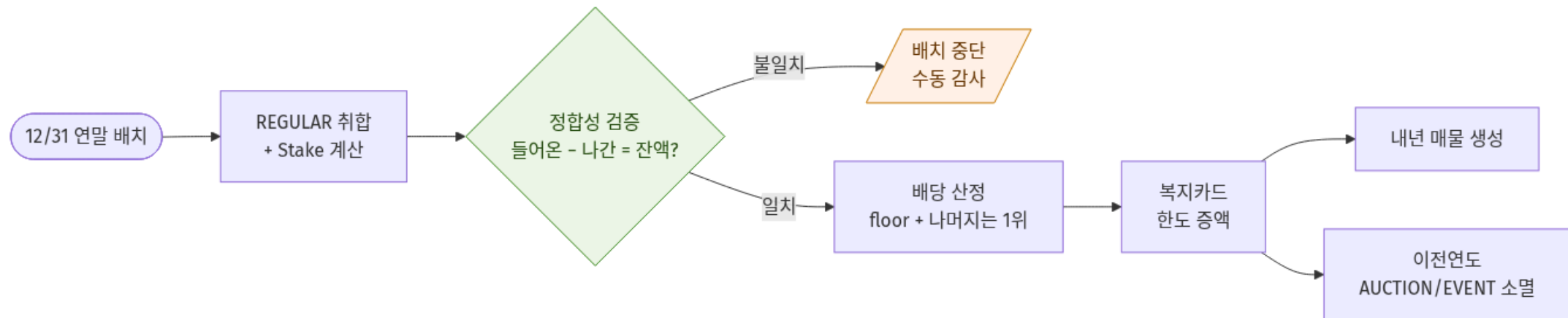
입찰 핵심 흐름 — write 락 → 단일 트랜잭션 → 실시간 알림

③-4 상태 다이어그램 — Auction 객체 생애주기



Auction 생애주기 — CREATED→OPEN→CLOSED→AWARDED/UNSOLD→EXPIRED

③-5 활동 다이어그램 — 연말 배치 (수익적립금 → 배당)



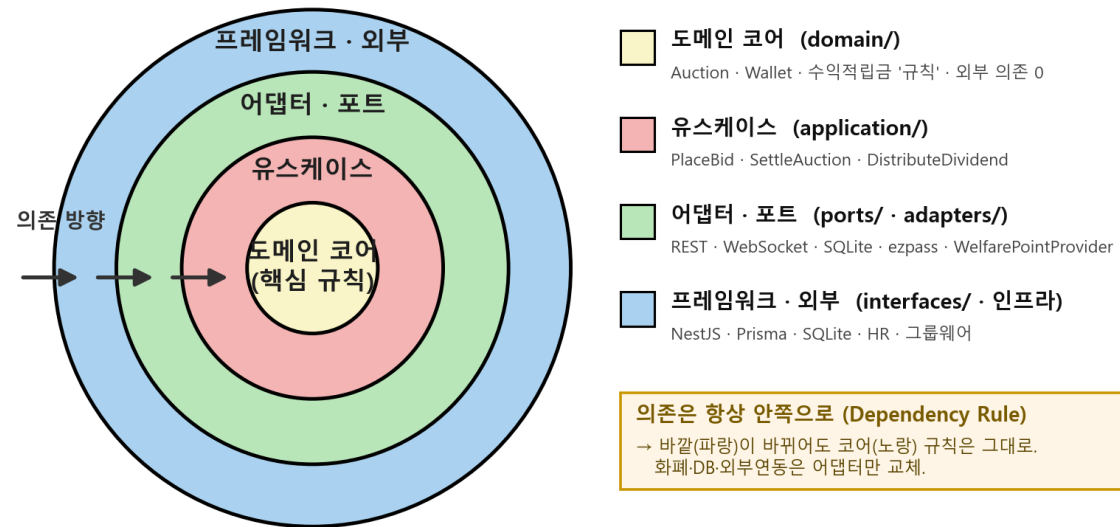
연말 배치 — 정합성 검증을 통과해야 배당, 실패 시 중단·수동 감사. · 전체 docs/uml

④ 소프트웨어 아키텍처

구조 선택 + 근거 + 블록 다이어그램

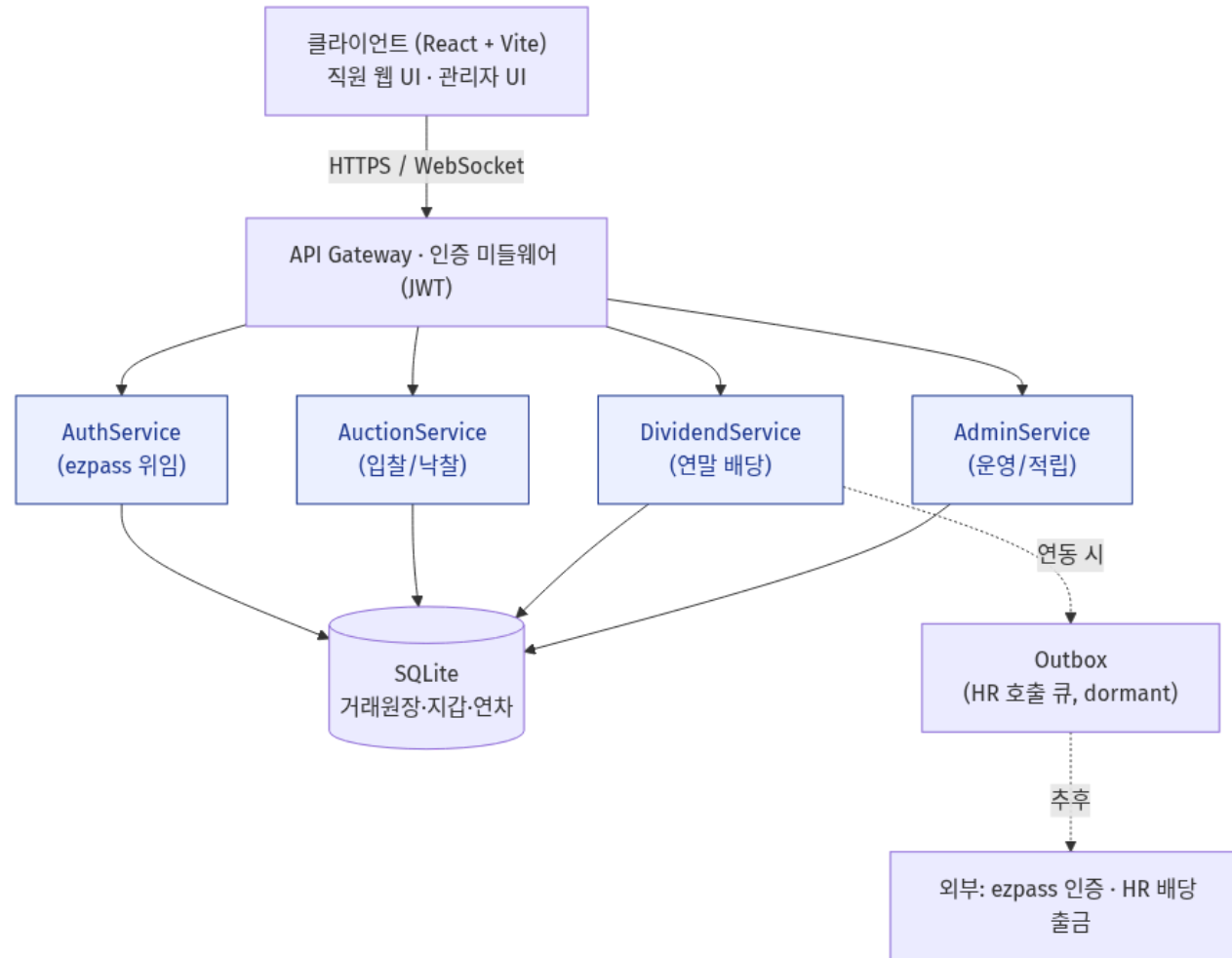
④ 아키텍처 — Ports & Adapters (헥사고날) + 클린 의존 규칙

헥사고날 / 클린 아키텍처 — 의존은 항상 안쪽으로



- 왜? 우리 시스템의 **핵심 도메인(규칙)**이 **외부 변화에 덜 흔들리게** 하려고 — DB·화폐·외부연동이 바뀌어도 **어댑터만 교체**, 코어는 그대로.
- **낙찰 정산 = 단일 트랜잭션**: 지갑 차감 + 적립 + 거래기록 + 연차부여 = All-or-Nothing.

④ 블록 다이어그램 — 논리 아키텍처 (MSA)



논리적으로 4개 서비스(MSA 구조), 실제 배포는 단일 앱.

⑤ 적용 디자인 패턴

GoF — Adapter · Observer (어디에 · 왜)

⑤ 핵심 패턴 — Adapter & Observer (코드에 실재)

Adapter (구조 패턴) — 헥사고날의 심장: "포트(약속) ← 어댑터(구현)"

- 화폐 지급 ← 복지포인트 어댑터(지갑 차감·적립) → 다른 화폐로 교체 가능
- 연차 부여 ← 내부 연차 어댑터 (→ 미래엔 그룹웨어 어댑터로 교체)
- 인증 ← **ezpass** 인증 어댑터
- 왜? 화폐·휴가·인증이 바뀌어도 **도메인 코어는 안 고침** — 어댑터만 교체

Observer (행위 패턴) — 부수효과 분리

- 입찰·낙찰이 일어나면 "**이벤트**"만 발행 → 구독자들이 각자 처리
- 구독자: **실시간 알림 · 통계 · 감사 로그**
- 왜? 새 부수효과를 추가해도 **핵심 로직(입찰/낙찰)은 안 고침** (개방-폐쇄)
- 구현: NestJS 이벤트 버스

⑥ 역할 분담 + 14주차 구현 계획

⑥ 현재 구현 현황 (13주차 시점)

설계에 그치지 않고 **헥사고날 골격 + 핵심 경로**까지 구현 착수 완료.

영역	상태	근거
헥사고날 구조	완료	domain / application / ports / adapters / interfaces 분리
도메인 코어	완료	Auction · Wallet · Ledger · Value Object(Point/UserId/AuctionId) + 단위테스트
경매 Use Case	완료	place-bid · settle-auction · create-auction 등 6종
인증	완료	ezpass 중앙 인증 위임 통합·검증 완료
DB	완료	SQLite 전환 (외부 서버 의존 제거, Prisma)
프론트엔드	부분	13개 화면(로그인·대시보드·경매·입찰·배당·관리자) 골격
연말 배치·배당	예정	14주차 핵심 작업

⑥ 역할 분담

2인 팀 → "주담당 + 부담당(크로스 리뷰)" 체계.

담당	주담당	핵심 산출물
김기철	기획·설계 · DB · 외부 연동	요구분석·SRS·UML·아키텍처(ADR) / DB 스키마·마이그레이션·무결성 트리거 / ezpass 인증·HR 연동 어댑터
오지석	전체 개발 · 디자인	백엔드 도메인 구현(경매·입찰·낙찰·배당), 프론트엔드 화면, UI/UX 디자인

- 의사결정: 기술 선택·주요 설계 결정은 **양자 합의**, 마이너 디테일은 담당 재량(PR 공유)
- 커뮤니케이션: GitHub PR(기술 논의 우선) · Issues(할 일) · 주간 스탠드업 30분
- 백업: 부재 시 부담당이 임시 주담당

⑥ 14주차 구현 계획 (마일스톤)

마일스톤	작업 · 완료 기준	담당
M-A 경매 핵심	입찰·낙찰 E2E — 동시 입찰 정합 100%	오지석 · 김기철(DB 락)
M-B 연차·차감	AUCTION→EVENT→REGULAR 강제 차감 통과	오지석
M-C 연말 배치·배당	풀 수집·Stake·배당 — 배당 합 = 수익적립금 검증	오지석 · 김기철(정합성)
M-D 관리자·UI	유찰 EVENT 지급 · 화면·디자인	오지석
M-E 외부 연동·시연	ezpass/HR 연동 점검 + E2E 데모	김기철 · 공동(시연)

리스크: ① 동시성(database is locked)→조기 부하테스트 ② 일학습병행 일정→가용시간 사전 공유 ③ 프론트-백 스펙 불일치→OpenAPI 기준.

마무리 — 핵심 메시지

1. **합법성 × 재무 안전성**을 동시에 푼 *B2E Escrow & Dividend* 설계
2. 요구사항을 **무결성 제약(DB-RULE)·NFR** 등식으로 못 박아 "정확성"을 검증 가능하게
3. **여러 UML 다이어그램**이 하나의 도메인 모델에서 일관 파생
4. **Hexagonal + GoF(Adapter·Observer)** 로 도메인 격리
5. 설계에 그치지 않고 **핵심 경로 구현 착수** → 14주차 시연으로 직결

감사합니다. 질문 환영합니다.

상세 설계 문서: [docs/](#) (SRS · UML · 설계 결정 22건 · ERD · OpenAPI)